

check out the source files from the Git repository and build them with NetBeans. You can use the following steps to do so:

- Download and install Git for Windows (<http://git-scm.com/download/win>)
- Launch the Git GUI and select *Clone existing repository*
- In the source location, enter the above mentioned URL for the Git repository and specify the target directory on your machine and click *Clone*.
- Once the repository is cloned, open it in NetBeans and build it. The build process will generate *.jar* files, which can then be included in any project.

However, if you think that's too much of work, I have uploaded a compiled version of the API at <http://goo.gl/Olc5MN>. Download the zip file, extract it and follow the next section.

Implementation

If you're using the GIT repository, copy the *simplevalidation-standalone-<version>.jar* file from *standalone/target* folder and *nbstubs-<version>.jar* from *nbstubs/target* folder to the project folder. However, if you have downloaded the compiled APIs from the above link, copy the two JAR files into the project folder.

Next, import the API in NetBeans. To do this, right click the project, select *Properties > Libraries > Add Jar/Folder*, specify the location of JAR files and press *OK*.

This API allows you to define the severity levels (FATAL, WARNING and INFO) of validation controls. The FATAL severity level is the strictest and requires the user to correct the input before submitting the form. The WARNING and INFO severity levels are a bit relaxed and can be used for validation of optional fields. It also decorates the Swing components (see Figure 1) according to the severity level set. The default is FATAL for all Swing components.

Sample code

A sample code snippet to enable validation in a login form with a username field, a password field and a login button is given below:

```
private ValidationPanel validationPanel;
private ValidationGroup validationGroup;

validationPanel = new ValidationPanel();
validationGroup = validationPanel.getValidationGroup();
validationPanel.validate();

/*Username field validation (Non-Empty String)*/

validationGroup.add(textField1,StringValidators.REQUIRE_NON_EMPTY_STRING);

/*Password field validation (Minimum Length 8, Non-Empty String, Disallowed Characters)*/

validationGroup.add(passwordField1,StringValidators.REQUIRE_NON_
```

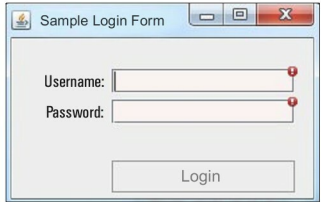


Figure 1: Login form

```
EMPTY_STRING,StringValidators.minLength(8),StringValidators.
disallowChars(new char[]{'\',' ','\'}));
validationGroup.addUI(new ValidationUpdate(button1));
```

```
/*Class to change the state of Login button depending on the
validity of input values and assigned severity*/
```

```
public class ValidationUpdate implements ValidationUI {
```

```
    JButton button,button2;
```

```
    public ValidationUpdate(JButton Button)
```

```
    {
        button=Button;
        button2=new JButton();
    }
```

```
    public ValidationUpdate(JButton Button, JButton Button2)
```

```
    {
        button=Button;
        button2=Button2;
    }
```

```
    public void clearProblem()
```

```
    {
        button.setEnabled(true);
        button2.setEnabled(true);
    }
```

```
    public void showProblem(Problem problem)
```

```
    {
        if(problem.isFatal())
        {
            button.setEnabled(false); button2.setEnabled(false);
        }
        else
        {
            button.setEnabled(true); button2.setEnabled(true);
        }
    }
```